

CSCE 637 COMPLEXITY THEORY · MODULE 11 · LECTURE NOTES

Hardness of Approximation

Exactly how well you can do, proved.

MODULE OUTCOMES

- Explain the gap technique and its connection to PCP.
- State the main inapproximability results.
- Explain the Unique Games Conjecture and its role.
- Explain why some problems have tight bounds and others do not.
- Use the approximability picture to choose an approach.

1 The gap technique

A GAP IN THE PROOF SYSTEM BECOMES A GAP IN THE OPTIMISATION PROBLEM

PCP says that a false statement's purported proof fails a random constant-size check with constant probability (Module 10 §3).

So encode an NP problem as an optimisation problem: 'satisfy as many of these checks as possible'. A true instance yields an assignment satisfying all of them; a false instance yields one satisfying at most a constant fraction. There are no instances in between.

And an approximation algorithm with a ratio better than that constant could distinguish the two cases — it would return a value above the false-instance ceiling exactly when the instance was true — which would decide the NP problem in polynomial time.

So no such approximation algorithm exists unless $P = NP$. The entire field of inapproximability is that one argument, applied through reductions that preserve the gap — which is why PCP created the field rather than merely contributing to it, and why almost no inapproximability results predate 1992.

Problem	Best achievable ratio	Hardness bound
Knapsack	$(1+\epsilon)$ for any ϵ , in time polynomial in $1/\epsilon$ — an FPTAS.	None. This is as good as approximation gets (CSCE 669 Module 10 §1's classes).
Euclidean TSP	$(1+\epsilon)$, a PTAS (Arora, Mitchell).	None.
Metric TSP	About 1.5 (Christofides, improved slightly in 2021).	No better than 123/122 — a large gap, and that is where the research is (§4).

Problem	Best achievable ratio	Hardness bound
MAX-3SAT	7/8, by a simple randomised assignment.	7/8 is optimal. TIGHT . So the trivial algorithm cannot be beaten.
MAX-CUT	0.878, by semidefinite programming (CSCE 669 Module 10 §3).	0.878 is optimal under the Unique Games Conjecture (§2).
Vertex cover	2, by the ten-line LP rounding.	No better than 2 under UGC; no better than about 1.36 unconditionally.
Set cover	$\ln n$, by the greedy algorithm.	$\ln n$ is optimal. TIGHT — the greedy algorithm cannot be improved.
Maximum clique	Essentially nothing useful.	No $n^{1-\epsilon}$ approximation unless $P = NP$. Inapproximable.

MAX-3SAT and set cover are the satisfying cases: the achievable ratio and the hardness bound coincide exactly, so the question is closed and the trivial algorithm is provably optimal — which is a rare and useful thing to be able to say, and it means any time spent looking for something better is provably wasted.

2 The Unique Games Conjecture

THE UNIQUE GAMES CONJECTURE, AND WHY SO MUCH RESTS ON IT

UGC asserts that a particular constraint-satisfaction problem — label assignment with permutation constraints — is hard to approximate, and from it follow optimal inapproximability results for a remarkable range of problems.

Under UGC, the Goemans–Williamson 0.878 ratio for MAX-CUT is optimal (CSCE 669 Module 10 §3) and vertex cover's factor of 2 is optimal — both of which look like artifacts of their particular analyses and turn out to be the true boundaries, which is strong circumstantial support for the conjecture.

And more strikingly: under UGC, a generic semidefinite programming relaxation achieves the optimal approximation ratio for every constraint satisfaction problem (Raghavendra) — so for that whole class the best algorithm is known and only the analysis of its ratio is missing. An unusual situation.

UGC is unproved and the evidence is mixed — subexponential-time algorithms for unique games exist, which is unusual for a problem conjectured hard and is the main reason for doubt. So state the conditionality when citing any of these results: 'optimal under UGC' is a weaker claim than 'optimal unless $P = NP$ ', and the difference matters.

3 Why some bounds are tight and others are not

- **Numeric problems with scalable precision get schemes.** Knapsack's FPTAS works by rounding the profit values to fewer significant digits and then solving exactly by dynamic programming — the structure permits trading precision for time continuously, which is exactly what a scheme requires.
- **Geometric problems get schemes,** because **space can be partitioned into cells and the error from treating each cell independently can be bounded** — which is how Euclidean TSP's PTAS works, and why the metric version (with no geometry) does not have one.
- **Constraint satisfaction gets constant factors,** and **the constant is usually set by how well a random assignment does** — a random assignment satisfies 7/8 of the clauses of a MAX-3SAT instance in expectation, **and that random baseline turns out to be optimal**, which is both satisfying and slightly deflating.
- **Covering problems get logarithms,** because **the greedy algorithm's harmonic-sum analysis is tight** — and the matching hardness result says the analysis was not merely the best available but the truth.
- **And problems where the optimum is a tiny structure resist approximation entirely** — **clique, because a clique that misses one vertex of a large clique may be tiny:** there is no partial credit. **The pattern across the list is about what an approximate answer is:** a 90%-good knapsack solution is nearly a knapsack solution, and a 90%-good clique is not nearly a clique.

4 Using the picture

LOOK UP BOTH SIDES BEFORE WRITING AN ALGORITHM

Find the best known approximation ratio and the best known hardness bound for your problem. If they coincide, the question is closed and you should implement the known algorithm rather than looking for a better one.

If there is a gap, the gap tells you how much improvement is possible — **metric TSP's gap between 1.5 achievable and 123/122 hard is where decades of research sits**, and knowing the gap tells you whether an improvement is worth attempting.

And if the hardness result says 'no constant factor', stop looking for one and move to Module 12's parameters, a restricted case, or a heuristic with a measured bound (CSCE 669 Module 13 §3).

Which takes twenty minutes against a reference — **and it is the same discipline as CSCE 627 Module 06 §4's prove-impossibility-first, with a quantitative answer instead of a binary one. The quantitative version is more useful**, because it tells you not only to stop but how much is available.

- **What your instances actually do.** A 2-approximation may routinely achieve 1.01 on real data, because **the guarantee is a worst case** (Module 01 §3) and the worst case may be nowhere near your inputs.
- **So measure the achieved ratio against a computed bound** rather than reporting the theoretical guarantee — which is CSCE 669 Module 13 §3's advice, and it turns a pessimistic claim into an accurate one.

- **And whether exact is affordable anyway. A solver may simply solve your instances exactly** (Module 04 §4), **making the whole approximation question moot** — which happens more often than the theory would lead you to expect.
- **Which is the recurring correction of this course: worst-case theory bounds what can be promised and not what will happen.**
- **So: look up the theory, then measure. Both, in that order** — the theory first because it may stop you wasting effort, and the measurement second because the theory does not describe your inputs. **Skipping either step is the common error**, and the two errors are made by different kinds of person.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Williamson & Shmoys — The Design of Approximation Algorithms (free PDF)	§1's table from the algorithm side , free in full, and the same book as CSCE 669 Module 10.
Arora & Barak — chapters 11 and 22 (free draft)	PCP's consequences and the gap technique of §1 , with the reductions.
Khot — On the Unique Games Conjecture (free survey)	§2, by the person who proposed it — including an honest account of the evidence against.
Raghavendra — Optimal Algorithms and Inapproximability Results for Every CSP? (free)	§2's striking consequence — the generic SDP being optimal under UGC.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Work through the gap argument** for one problem, from PCP to inapproximability.
- 2 **Implement the random-assignment 7/8 algorithm for MAX-3SAT** and measure its achieved ratio.
- 3 **Confirm that 7/8 is the expected value** by direct calculation.
- 4 **Implement greedy set cover** and measure its ratio against the LP bound on random instances.
- 5 **Try to beat it** and relate your failure to the tightness result.
- 6 **Look up both sides for three problems from your own work** and record the gap in each.
- 7 **Identify which of your problems has a closed question** and implement the optimal algorithm.
- 8 **Measure a 2-approximation's achieved ratio on real data** and compare against the guarantee.

- 9 **Classify five problems by §3's patterns** and check the classification against the known results.
- 10 **Find a result cited as 'optimal' that is conditional on UGC** and note whether the citation says so.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Explain how a PCP gap becomes an inapproximability result.
- 2 Why did PCP create the field rather than contribute to it?
- 3 Give the approximability picture for eight problems.
- 4 Which two are tight, and what does that mean practically?
- 5 State UGC and three results that depend on it.
- 6 Why is the evidence for UGC mixed?
- 7 Give five patterns determining approximability.
- 8 What is the underlying principle behind all five?
- 9 Give the four-step procedure for using the picture.
- 10 Name two things the picture does not tell you.